Web Systems Development

The login lab, file by file

Session 22 · Fri 11/13 · From the midterm prototype to a real login, and how the Laravel React starter kit does the same job

RPI

By the end of class, your app knows who is using it

BEFORE · THE MIDTERM

- The header says "Acting as Alex Rivera".
- Adding ?as=priya to the address makes you Priya. Nobody typed a password.
- A fake server in the browser decides everything, so anyone can change the answer.

AFTER · ON THE PAGES

- /register makes an account and logs you in.
- /login checks a password. A wrong one shows a message under the field.
- The header shows your name and a Log out button. A refresh keeps you logged in.

AFTER · ON THE SERVER

- Laravel checks the password and keeps the login in a session cookie.
- Logged out, GET /api/user answers 401.
- Twenty-one bad logins in a minute answer 429.

BUILDS ON TUESDAY

Tuesday named the two questions: who are you (authentication) and what may you do (authorization). Today builds the first one, with the session cookie from Part 2 of that lecture. The second one, the policy, is Homework 9 items 3 and 4.

Nine files, in this order, with two pauses

1	`api/app/Models/User.php`	The password is stored as a hash. role cannot arrive in a request.
2	`api/app/Http/Controllers/Api/AuthController.php`	New: register(), login(), logout(), user().
3	`api/routes/api.php`	Four account routes. Your three writes move inside auth:sanctum.
4	`api/app/Providers/AppServiceProvider.php`	The limit on login tries.
5	`api/bootstrap/app.php, docker-compose.yml`	Turn the session on for /api, and name your web port.
6	`web/src/api.js`	Four functions: currentUser, login, register, logout.
7	`web/src/session.jsx`	The context that knows who is logged in gains three actions.
8	`web/src/pages/AuthPages.jsx`	New: the Log in and Register forms. Two routes in App.jsx.
9	`web/src/components/Layout.jsx`	The header: your name and Log out, or a Log in link.

PAUSE 1 · AFTER FILE 3

route:list shows login, register, user and logout.

PAUSE 2 · AFTER FILE 5

check-login.sh: 9 of 9.

FILES 6 TO 9

The pages. This is what continues as Homework 9 item 2.

PART 1 OF 4

Two finished apps to read

Nobody invents a login. You read one that works and adapt it.

We have two: the Laravel React starter kit, and the pantry example.

1

React draws the screen. The server keeps the data.

FRONTEND · REACT, IN THE BROWSER

A component is a function: data in, screen out.

- What is on the screen
- What a click or a key press does
- When the data changes, React redraws

HTTP · THE ONLY CONNECTION

request: a method, an address, the cookie





response: a status code, and JSON

BACKEND · LARAVEL, ON THE SERVER

It keeps the data and enforces the rules.

- The database
- Who is logged in
- What each person may read or change

ONE CLICK, FIVE STEPS

1. You press Log in.
2. React sends POST /api/login.
3. Laravel checks the password and answers 200 with JSON.
4. React puts the user in state.
5. React redraws. Your name is in the header.

Why the rules are on the server: anyone can change what runs in their own browser, so a rule that exists only in React is a suggestion. The midterm's ?as=priya address showed it.

Two shapes: one app, or an API with a separate React app

STARTER KIT · ONE APP

Browser. Inertia's React adapter resolves page components from resources/js/pages and updates the view without a full reload.

Inertia visit: GET /login

Inertia response: page + props

Laravel. routes/web.php decides the page. Inertia packages the page name and props; Fortify checks the password.

One codebase and one deploy. Inertia is the bridge between Laravel routes/controllers and React pages.

YOUR PROJECT · TWO APPS

Browser. Your React app in web/src. React Router decides the page.

fetch /api/login

JSON: data, or errors

Laravel. routes/api.php answers JSON. Your AuthController checks the password.

Two codebases. Any client can use the API: a phone app, another site, curl.

Both keep the login in a session cookie, and both hash the password the same way. The rules are the same; the plumbing between React and Laravel is different.

Yes, the starter kit has a React side. It has no API.

STARTER KIT `resources/js/`

```
resources/js/
  app.tsx            # the entry point, like your main.jsx
  pages/             # one React component per screen
    auth/login.tsx
    auth/register.tsx
    dashboard.tsx
    settings/profile.tsx
  layouts/           # the frame around a page, like Layout.jsx
  components/        # buttons, inputs, the user menu
  hooks/
  routes/            # generated from Laravel's routes
  actions/           # generated: do not edit
```

WHAT YOU WILL NOT FIND, AND WHY

routes/api.php	Laravel sends each page its data as props.
api.js, fetch()	Inertia makes the requests.
React Router	Laravel's routes/web.php decides the page.
session.jsx	The user arrives as a prop on every page.
A login controller	A package, Fortify, has it.
A policy	No resource in the kit belongs to one user.

The files are .tsx: TypeScript. Read past the type annotations; the React is the React you know.

Inertia: Laravel answers with a page's name and its props

STARTER KIT app/Providers/FortifyServiceProvider.php

```
GET /login: no HTML is written here
Fortify::loginView(fn (Request $request) =>
    Inertia::render('auth/login', [
        'canResetPassword' => true,
        'status' => $request->session()->get('status'),
    ]));
```

WHAT THE BROWSER RECEIVES (shortened)

```
{
  "component": "auth/login",
  "props": {
    "canResetPassword": true,
    "auth": { "user": null },
    "errors": {}
  },
  "url": "/login"
}
```

1 · A LARAVEL ROUTE

The route does not return HTML or a Blade view. It names a React component, auth/login, and gives it props.

2 · INERTIA IN THE BROWSER

Inertia finds resources/js/pages/auth/login.tsx and renders it with those props.

3 · THE NEXT CLICK

An Inertia <Link> sends the request with the header X-Inertia: true. Laravel answers with only this JSON, and Inertia swaps the component. The page does not reload.

Demo: a Students page, from the route to the controller

STARTER KIT routes/web.php

```
Route::middleware(['auth', 'verified'])->group(function () {
    Route::inertia('dashboard', 'dashboard')->name('dashboard');
    Route::get('students', [StudentController::class, 'index'])
        ->name('students.index');
});
```

STARTER KIT app/Http/Controllers/StudentController.php · new file

```
class StudentController extends Controller
{
    public function index()
    {
        return Inertia::render('students/index', [
            'students' => [
                ['id' => 1, 'name' => 'Alex Rivera', 'major' => 'ITWS'],
                ['id' => 2, 'name' => 'Priya Shah', 'major' => 'CSCI'],
            ],
        ]);
    }
}
```

1 · THE ROUTE

An address and a method, pointed at a controller method. It is inside the auth group, so a logged-out visitor is sent to /login.

2 · THE CONTROLLER

It gets the data and names a page. The data is typed in here; in a real app it is an Eloquent query, as in session 15.

3 · NO HTML, NO API

The controller returns neither. Inertia turns this into the page object: the name students/index, and the students as props.

MAKE THE FILE

php artisan make:controller StudentController

The page: props in, components out

STARTER KIT `resources/js/pages/students/index.tsx` · new file

```
import { Badge } from '@/components/ui/badge';
import { Card, CardContent, CardHeader, CardTitle }
    from '@/components/ui/card';

type Student = { id: number; name: string; major: string };

export default function Students({ students }: { students: Student[] }) {
    return (
        <div className="grid gap-4 p-4 md:grid-cols-3">
            {students.map((student) => (
                <Card key={student.id}>
                    <CardHeader>
                        <CardTitle>{student.name}</CardTitle>
                    </CardHeader>
                    <CardContent>
                        <Badge>{student.major}</Badge>
                    </CardContent>
                </Card>
            ))}
        </div>
    );
}
```

PROPS FROM LARAVEL

students is the array the controller gave Inertia::render(). There is no fetch, no useEffect and no loading state: the data is there the first time the component runs.

THE TYPE

TypeScript's description of one student. It has to match what the controller sends. Nothing checks that for you.

CARD AND BADGE

shadcn/ui components. They are files in this project, under resources/js/components/ui. Next slide.

THE REACT YOU KNOW

.map() with a key, props and JSX, from sessions 8 to 11.

shadcn/ui gives you the source. Then watch the round trip.

SHADCN/UI: COMPONENTS YOU OWN

```
$ npx shadcn@latest add table
writes resources/js/components/ui/table.tsx
```

- It copies the component's source into your project. It is not a dependency in node_modules.
- So you can open table.tsx, read it, and change it. It is React and Tailwind classes, as in sessions 6 to 9.
- The cost: updates do not arrive by themselves. The file is yours to keep up.

THE ROUND TRIP, IN DEVTOOLS · NETWORK

1. **docker compose watch, then open localhost:8040/students**

   One request of type document. The answer is HTML, with the page object inside it.

2. **Click Dashboard, then Students, in the sidebar**

   The request carries X-Inertia: true. The answer is JSON: component and props.students.

3. **Change a name in the controller. Click Students again.**

   The new name is on the screen. You edited one PHP file and no React.

PREDICT Before step 2: the first visit was answered with HTML. Will the second visit to /students be HTML or JSON?

The same six jobs, done two ways

THE JOB	STARTER KIT · INERTIA	YOUR PROJECT · API/ AND WEB/
Which page is this address?	Laravel: routes/web.php	React Router: App.jsx
How does a page get its data?	Props, sent by the Laravel route	fetch('/api/...') in api.js
How does a form submit?	Inertia's <Form> posts to a Laravel route	onSubmit calls a function in api.js
Where do the 422 messages arrive?	An errors prop that Inertia fills in	The object api.js throws: err.errors
Who is logged in?	A prop on every page: auth.user	GET /api/user, kept by session.jsx
Who checks the password?	A package: Fortify	Your AuthController::login()

ASK Which column has less code for you to write? Which column can a phone app use?

Another integrated path: Next.js starts from React

INERTIA · BUILDS ON LARAVEL

Browser. Inertia resolves React pages from resources/js/pages and swaps them without a full reload.

GET /students

page + props

Laravel. routes/web.php and controllers choose the page and data; Inertia sends the page name and props.

Backend-first. In this starter kit, Inertia builds on Laravel: Laravel keeps routing and React draws its pages.

NEXT.JS · BUILDS ON REACT

Browser. React pages and layouts come from files under app/.

GET /students

HTML / React

Next.js. app/students/page.tsx becomes /students; the directory structure defines the route.

Frontend-first. Next.js builds on React: React is the centre, and Next.js adds routing and server work.

Both make one integrated React app. The difference is the starting point: Inertia extends Laravel; Next.js extends React.

Three ways to connect React to a server

THE QUESTION	YOUR PROJECT · API + REACT	INERTIA · THE STARTER KIT	NEXT.JS
Who decides the page for an address?	React Router, in the browser	Laravel: routes/web.php	Next.js, from the folders under app/
Who gets the data?	Your React code: fetch('/api/…')	A Laravel controller, sent as props	The component itself, running on the server
Where does React render?	In the browser	In the browser	On the server first, by default
Is there an API to write?	Yes: your api/	No	Only if another client needs one
What is it built around?	Two halves and a contract	The backend: Laravel, with React as its views	The frontend: React, with a server added

ASK A team with a working Laravel app wants React pages. Inertia or Next.js? Why?

Backend-first or frontend-first. Inertia extends Laravel: the backend stays in charge and React draws its pages. Next.js extends React: React is the centre and the server work happens inside it.

One app or two? Five things decide

WHAT YOU LOOK AT	LEANS TO ONE INTEGRATED APP	LEANS TO AN API AND A SEPARATE REACT APP
The team	A small team where everyone works on both sides	Frontend and backend specialists, each shipping on their own schedule
The clients	One website	A phone app or a partner needs the same data, so the API exists anyway
Deploying	One deploy. Both sides release together.	The React app sits on a CDN and is released and scaled by itself
Coordination	Nothing to agree between the two sides	A contract to keep: versions, CORS, and a login across two addresses
Interactivity	Content, lists and forms	App-like screens: offline use, a rich editor

Conway's Law: a system ends up shaped like the organisation that built it. Two teams tend to produce two apps.

ASK Your project is the right-hand shape. Which rows have you already paid for, and in which homework?

SUMMARY

Part 1: the rules lift across; the plumbing does not

1	**React and the server**	React draws the screen from data. The server keeps the data and the rules. HTTP is the only connection.
2	**Inertia**	A Laravel route names a React component and its props. No API, no client-side router.
3	**What lifts**	The hashed password, the allow-list, the registration rules, the rate limiter, the three lines of log out.
4	**What does not**	How a form reaches the server. Yours uses fetch, so that part comes from the pantry.

Next: Part 2, the five API files. Open your team's api/ folder.

PART 2 OF 4

The login, on the API

2

Five files. The starting point is the api/ you built in Homework 7.

The midterm pantry had no api/ at all: a fake server in the browser answered.

User.php: store a hash, and keep role out of reach

FILE `api/app/Models/User.php`

```php
#[Fillable(['name', 'email', 'password'])]
#[Hidden(['password', 'remember_token'])]
class User extends Authenticatable
{
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }

    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }
}
```

THE HASHED CAST

Whenever password is set, Laravel stores a bcrypt hash. The column never holds the password itself, so a stolen database does not hand over logins.

THE ALLOW-LIST

Only the names in #[Fillable] can be filled from a request. role is not there, so "role": "admin" in a request body is ignored.

FROM THE STARTER KIT

Its app/Models/User.php has the same two lines. Nothing changes; you add isAdmin().

AuthController: login() is three steps

FILE api/app/Http/Controllers/Api/AuthController.php

```
public function login(Request $request)
{
    $credentials = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required', 'string'],
    ]);

    if (! Auth::attempt($credentials)) {
        throw ValidationException::withMessages([
            'email' => 'These credentials do not match our records.',
        ]);
    }
    $request->session()->regenerate();

    return ['data' => $request->user()];
}
```

1 · VALIDATE

A missing email or password is a 422 before anything else runs.

2 · ATTEMPT

Auth::attempt() finds the user by email and compares the password with the stored hash. One message for a wrong password and for an unknown email, so the form does not reveal which emails have accounts.

3 · REGENERATE

A new session id after logging in, so an id an attacker planted before the login is worthless.

IN THE STARTER KIT

Fortify runs the same three steps, inside the package. There is no file in the kit to read.

Fortify: what it is, and should your team use it?

WHAT FORTIFY IS

A Laravel package the docs call "a frontend agnostic authentication backend". It adds the routes and controllers for log in, register, password reset, email verification and two-factor. It has no pages: you bring the forms.

FORTIFY IS NOT A REPLACEMENT FOR SANCTUM

Sanctum answers "is this request logged in?" That is auth:sanctum on your routes. Fortify supplies the routes that log someone in. An API with a React app can use both, and the Laravel docs say so.

1

Write it by hand

HOMEWORK 9

AuthController.php, about 70 lines. You can read and explain every line, and the check script expects exactly its replies. You get log in, register and log out, and nothing more.

2

Fortify in your api/

AN OPTION FOR THE FINAL PROJECT

Install it and turn its pages off. POST /login, /register and /logout exist without a controller, and so do password reset and email verification. You still write the React pages and GET /api/user, and you keep Sanctum.

3

The starter kit

A DIFFERENT PROJECT SHAPE

Fortify plus finished pages, with Inertia. Everything is done on day one, but it replaces your web/ and your API with one app.

For Homework 9, write it by hand: the login is the thing being taught, and it is 70 lines. Reach for Fortify when your project needs password reset or email verification.

register() and logout(): both lift from the starter kit

FILE AuthController.php · register()

```
public function register(Request $request)
{
    $fields = $request->validate([
        'name' => ['required', 'string', 'max:80'],
        'email' => ['required', 'email', 'unique:users'],
        'password' => ['required', 'confirmed',
            Password::min(8)],
    ]);
    $user = User::create($fields)->fresh();
    Auth::login($user);
    $request->session()->regenerate();
    return response()->json(['data' => $user], 201);
}
```

FROM THE STARTER KIT

app/Actions/Fortify/CreateNewUser.php validates the same three fields and calls User::create() with the same three names. confirmed means the body must also carry password_confirmation. fresh() reloads the user, so the reply includes the role the database filled in.

FILE AuthController.php · logout()

```
public function logout(Request $request)
{
    Auth::guard('web')->logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return response()->noContent();   // 204
}
```

FROM THE STARTER KIT

app/Http/Controllers/Settings/ProfileController.php, in destroy(): the same three lines. Forget the user, throw the session away, and issue a new CSRF token. Yours answers 204; the kit redirects to the home page.

routes/api.php: what is open, and what needs a login

FILE api/routes/api.php

```
Route::get('/items', [ItemController::class, 'index']);
Route::get('/items/{item}', [ItemController::class, 'show']);

Route::post('/login', [AuthController::class, 'login'])
    ->middleware('throttle:login');
Route::post('/register', [AuthController::class, 'register'])
    ->middleware('throttle:register');

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', [AuthController::class, 'user']);
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::post('/items', [ItemController::class, 'store']);
    Route::put('/items/{item}', [ItemController::class, 'update']);
    Route::delete('/items/{item}', [ItemController::class, 'destroy']);
});
```

OPEN TO ANYONE

The two reads, and the two routes a person must reach before they have a login.

AUTH:SANCTUM

The guard from Tuesday. It looks for a login on the request and answers 401 when there is none. The controller never runs.

YOUR THREE WRITES

They move inside the group. They may keep answering 501 today; Homework 9 item 3 fills them in.

PAUSE 1

docker compose exec app php artisan route:list --path=api shows login, register, user, logout.

The login limiter: the starter kit's, with one word changed

STARTER KIT app/Providers/FortifyServiceProvider.php

```
RateLimiter::for('login', function (Request $request) {
    $throttleKey = Str::transliterate(
        Str::lower($request->input(Fortify::username()))
        .'|'.$request->ip());

    return Limit::perMinute(5)->by($throttleKey);
});
```

YOURS api/app/Providers/AppServiceProvider.php · boot()

```
RateLimiter::for('login', function (Request $request) {
    $throttleKey = Str::transliterate(
        Str::lower($request->input('email'))
        .'|'.$request->ip());

    return Limit::perMinute(5)->by($throttleKey);
});
RateLimiter::for('register', fn (Request $request) =>
    Limit::perMinute(60)->by($request->ip()));
```

WHAT IT DOES

Five tries a minute for one email from one address. The sixth answers 429 Too Many Requests.

WHY THE EMAIL IS IN THE KEY

Keyed on the address alone, one person's typos would lock out everyone on the same dorm network.

THE ONE CHANGE

Fortify::username() is the name of the login field. Your project has no Fortify, so write 'email'. Add use Illuminate\Support\Str; at the top.

Two lines the starter kit does not need, and you do

FILE api/bootstrap/app.php

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->statefulApi();
})
```

FILE docker-compose.yml

```
app:
  environment:
    SANCTUM_STATEFUL_DOMAINS: localhost:8010,web:5173
```

WHY THE STARTER KIT DOES NOT NEED THEM

In the starter kit every route is a web route, and web routes always have a session and CSRF protection. Your routes are in routes/api.php, and /api routes have neither by default: an API normally serves clients that carry a token. statefulApi() gives /api a session when the request comes from your own pages, and SANCTUM_STATEFUL_DOMAINS says which addresses count as your own pages.

USE YOUR OWN PORT

The pantry's compose file says 5181. Yours is 8010, or whatever your web port is. Restart docker compose watch after editing the compose file.

WHEN IT IS MISSING

Login answers 500 "Session store not set on request", or login works and the next request is 401.

Run the check: nine requests, PASS or FAIL for each

FROM YOUR REPO'S ROOT pass your own resource path in place of items

```
$ bash ../itws2110-course/inclass/wk12/check-login.sh http://localhost:8010 items
PASS  1. POST /api/register -> 201, and the extra "role":"admin" was ignored
PASS  2. POST /api/login with the wrong password -> 422
PASS  3. the wrong-password and unknown-email messages are identical
PASS  4. POST /api/login -> 200 and a session cookie
PASS  5. GET /api/user with the cookie -> 200, with the email
PASS  6. GET /api/user without a cookie -> 401
PASS  7. POST /api/items without a cookie -> 401 (not 501)
PASS  8. POST /api/logout -> 204, and GET /api/user is 401 again
PASS  9. twenty-one bad logins -> 429 (throttle:login)
9 passed, 0 failed
```

ASK Match each check to the file that makes it pass. Which file is check 1? Check 3? Check 7? Check 9?

A FAIL prints the status code and Laravel's message. Read that line first, then the Troubleshooting table on the lab page.

SUMMARY

Part 2: five files, and the server now knows who is asking

1	**User.php**	Passwords are hashes. role cannot come from a request.
2	**AuthController.php**	login() validates, attempts, regenerates. One message for every failed login.
3	**routes/api.php**	Reads and the two account routes are open. Everything else is inside auth:sanctum.
4	**Provider, bootstrap, compose**	Five tries a minute. A session for /api, for requests from your own pages.

Next: Part 3, the pages. Open project/pantry-example-midterm/web/src.

PART 3 OF 4

The login, on the pages

3

Four files. The starting point is the midterm prototype's web/ folder. Every green line is a line the midterm did not have.

First, the midterm has to talk to a real server

The midterm prototype answered every request from fake-server.js, in the browser. Your team made these three changes in Homework 7 and 8. The pantry demo makes them now.

MIDTERM web/src/api.js · request()

```
async function request(method, path, body) {
  await wait(DELAY_MS);
  const response = server.handle(method, path, body);
  if (response.status >= 400)
    throw { status: response.status, ...response.body };
  return response.body;
}
```

FINAL web/src/api.js · request()

```
async function request(method, path, body) {
  const headers = { Accept: 'application/json' };
  if (method !== 'GET')
    headers['X-XSRF-TOKEN'] = await csrfToken();
  const res = await fetch(path, {
    method, headers, credentials: 'include',
    body: body && JSON.stringify(body),
  });
  const data = await res.json().catch(() => null);
  if (!res.ok) throw { status: res.status, ...data };
  return data;
}
```

WHAT STAYS THE SAME

A failed request still throws { status, message, errors }. That is why no page had to change when the server became real.

Also: vite.config.js forwards /api and /sanctum to Laravel, and docker-compose.yml gains the app and db services. Both shortened here; copy them from the final.

api.js: four functions, one per account route

FILE web/src/api.js

```
already there at the midterm
export const currentUser = () =>
  request('GET', '/api/user').then((body) => body.data);

export const login = ({ email, password }) =>
  request('POST', '/api/login', { email, password })
    .then((body) => body.data);

export const register = (fields) =>
  request('POST', '/api/register', fields)
    .then((body) => body.data);

export const logout = () =>
  request('POST', '/api/logout').then(() => undefined);
```

ONE FUNCTION PER ROUTE

Each one names a method and a path, and unwraps data from the reply. A page never calls fetch itself.

ERRORS PASS THROUGH

A 422 from login is thrown by request() as { status: 422, errors: { email: [...] } }. The form on the next slides catches it.

IN THE STARTER KIT

There is no api.js. The address /login is in a generated file, and Inertia sends the request. Part 4 shows it.

session.jsx: the context gains three actions

FILE web/src/session.jsx

```
export function SessionProvider({ children }) {
  const [user, setUser] = useState(null);
  const [status, setStatus] = useState('checking');
  useEffect(() => { /* unchanged: GET /api/user, once */ }, []);

  const value = {
    user,
    status,
    async login(credentials) {
      const u = await api.login(credentials); setUser(u); return u;
    },
    async register(fields) {
      const u = await api.register(fields); setUser(u); return u;
    },
    async logout() { await api.logout(); setUser(null); },
  };
  return (
    <SessionContext.Provider value={value}>
      {children}
    </SessionContext.Provider>
  );
}
```

AT THE MIDTERM

The value was { user, status }. Every page could read who was logged in; none could change it.

NOW

Each action calls the API, then calls setUser. Changing state re-renders every component that calls useSession(), so the header updates without a reload.

WHY A CONTEXT

The header, the shelf and the item page all need the user. A context hands it to whoever asks.

IN THE STARTER KIT

No context. The user is a prop Laravel sends with every page: usePage().props.auth.

AuthPages.jsx: one hook puts a 422 next to its field

FILE `web/src/pages/AuthPages.jsx · new file`

```
function useAuthForm(initial, submit) {
  const [values, setValues] = useState(initial);
  const [errors, setErrors] = useState({});
  const [saving, setSaving] = useState(false);
  const change = (e) =>
    setValues((prev) => ({ ...prev, [e.target.name]: e.target.value }));

  async function handleSubmit(e) {
    e.preventDefault();
    setSaving(true);
    try {
      await submit(values);
    } catch (err) {
      setErrors(err.status === 422
        ? err.errors
        : { form: [err.status === 429
            ? 'Too many tries. Wait a minute and try again.'
            : err.message] });
      setSaving(false);
    }
  }
  return { values, errors, saving, change, handleSubmit };
}
```

THREE PIECES OF STATE

values: what is typed. errors: the server's messages, by field. saving: true while the request is out, to disable the button.

A 422

err.errors is { email: ["These credentials..."] }. Storing it as is means errors.email goes straight to the email field.

ANYTHING ELSE

A 429 or a 500 has no field, so it goes under the key form and is shown above the button.

IN THE STARTER KIT

Inertia's <Form> keeps all three for you: it hands the page processing and errors.

AuthPages.jsx: the Log in page

FILE `web/src/pages/AuthPages.jsx · shortened`

```
export function LoginPage() {
  const { login } = useSession();
  const navigate = useNavigate();
  const form = useAuthForm(
    { email: '', password: '' },
    async (values) => { await login(values); navigate('/'); },
  );

  return (
    <form onSubmit={form.handleSubmit} noValidate>
      <Field id="email" label="Email" type="email"
        autoComplete="username" value={form.values.email}
        onChange={form.change} errors={form.errors.email} />
      <Field id="password" label="Password" type="password"
        autoComplete="current-password" value={form.values.password}
        onChange={form.change} errors={form.errors.password} />
      {form.errors.form && <p role="alert">{form.errors.form[0]}</p>}
      <Button type="submit" disabled={form.saving}>Log in</Button>
    </form>
  );
}
```

ON SUCCESS

login() comes from the session, so it also sets the user. Then navigate('/') goes to the shelf.

FIELD

The midterm's component: a label, an input, and errors[0] underneath. The same one the item form uses.

AUTOCOMPLETE

Tells the browser and password managers which field is which.

REGISTER

The same shape with four fields: name, email, password, password_confirmation.

THEN, IN APP.JSX

Two routes: login and register. Next slide.

App.jsx gets two routes; the header shows who is logged in

FILE web/src/App.jsx

```
import { LoginPage, RegisterPage }
  from './pages/AuthPages.jsx';

<Route element={<Layout />}>
  <Route index element={<ShelfPage />} />
  <Route path="items/:id" element={<ItemPage />} />
  <Route path="login" element={<LoginPage />} />
  <Route path="register" element={<RegisterPage />} />
</Route>
```

AT THE MIDTERM, THE HEADER SAID

`Acting as {user.name}` and ?as=priya in the address changed who that was.

FILE web/src/components/Layout.jsx · shortened

```
const { user, logout } = useSession();
const navigate = useNavigate();
async function handleLogout() {
  await logout();
  navigate('/');
}

{user ? (
  <>
    <span>{user.name}</span>
    <Button onClick={handleLogout}>Log out</Button>
  </>
) : (
  <Link to="/login">Log in</Link>
)}
```

WHY IT UPDATES BY ITSELF

Layout calls useSession(). When login() or logout() calls setUser, Layout re-renders with the new user.

Try it: register, log out, log in, refresh

PREDICT You are logged in as Alex and press the browser's refresh button. React's state is gone. Are you still logged in?

1	**Register a new account**	Your name appears in the header. No reload happened.
2	**Register the same email again**	"That email already has an account." under the email field.
3	**Log out, then log in with a wrong password**	One message under the email field. The same one for an email that does not exist.
4	**Log in as alex@example.edu / password, then refresh**	Still Alex.

Why the refresh keeps you logged in: the login is not in React. It is a session cookie in the browser. When the app loads, SessionProvider asks GET /api/user, the browser sends the cookie, and Laravel answers with Alex.

SUMMARY

Part 3: four files, and the pages now use the real login

1	**api.js**	One function per account route. A failed request throws { status, errors }.
2	**session.jsx**	login, register and logout call the API, then setUser. Every page re-renders.
3	**AuthPages.jsx**	A hook holds values, errors and saving. A 422 lands next to its field.
4	**App.jsx, Layout.jsx**	Two routes. The header shows a name and Log out, or a Log in link.

Next: Part 4. The same four jobs in the starter kit, where Inertia does them.

PART 4 OF 4

Reading the starter kit's React side

4

You just built four files. The starter kit needs none of them.

Read how it does the same four jobs. Nothing here is typed into your project.

app.tsx: Laravel names the page; this picks the frame

STARTER KIT resources/js/app.tsx · shortened

```
createInertiaApp({
    title: (title) => (title ? `${title} - ${appName}` : appName),
    layout: (name) => {
        switch (true) {
            case name === 'welcome':
                return null;
            case name.startsWith('auth/'):
                return AuthLayout;
            case name.startsWith('settings/'):
                return [AppLayout, SettingsLayout];
            default:
                return AppLayout;
        }
    },
    strictMode: true,
});
```

CREATEINERTIAAPP

Starts React, like createRoot().render() in your main.jsx. Inertia then renders whichever page component Laravel names.

NO LIST OF ROUTES

Your App.jsx lists every address. Here the addresses are in Laravel's routes/web.php, and a page is found by its file name under pages/.

LAYOUT

name is the page name, such as auth/login. Pages under auth/ get the centred card; everything else gets the sidebar.

IN YOURS

<Route element={<Layout />}> wraps the pages in App.jsx.

login.tsx: Inertia's <Form> holds the state your hook holds

STARTER KIT resources/js/pages/auth/login.tsx · shortened

```
export default function Login({ status, canResetPassword }: Props) {
    return (
        <Form {...store.form()} resetOnSuccess={['password']}>
            {({ processing, errors }) => (
                <>
                    <Label htmlFor="email">Email address</Label>
                    <Input id="email" type="email" name="email"
                        required autoComplete="email" />
                    <InputError message={errors.email} />
                    <Label htmlFor="password">Password</Label>
                    <PasswordInput id="password" name="password"
                        required autoComplete="current-password" />
                    <InputError message={errors.password} />
                    <Button type="submit" disabled={processing}>
                        Log in
                    </Button>
                </>
            )}
        </Form>
    );
}
Login.layout = { title: 'Log in to your account' };
```

PROPS FROM LARAVEL

status and canResetPassword are the props from Inertia::render() in Part 1.

A FUNCTION AS CHILDREN

<Form> calls this function and hands it processing and errors. Those are your hook's saving and errors.

NO VALUE, NO ONCHANGE

The inputs only have a name. <Form> reads them when it submits, like a plain HTML form.

ERRORS.EMAIL

A string, the first message. In yours it is an array, so Field shows errors[0].

STORE.FORM()

Where to post. Next slide.

The same form, two ways

THE JOB	STARTER KIT · LOGIN.TSX	YOURS · AUTHPAGES.JSX
What is typed	Stays in the inputs; read at submit	useState: values, updated on every key
Sending it	<Form> posts to store.form()	handleSubmit calls login() in api.js
The 422 messages	errors, handed to the children function	catch (err), then setErrors(err.errors)
While it is sending	processing	saving
After success	Laravel redirects; Inertia follows	navigate('/')
The CSRF token	Inertia sends it	request() in api.js sends it

NOTICE The server's answer to a wrong password is the same in both: 422, with a message keyed on email. What differs is who catches it.

store.form() is generated from Laravel's route list

STARTER KIT resources/js/routes/login/index.ts · generated, shortened

```
/**
 * @see ...\Fortify\Http\Controllers\AuthenticatedSessionController::store
 * @route '/login'
 */
export const store = (options) => ({
    url: store.url(options),
    method: 'post',
});
store.definition = { methods: ['post'], url: '/login' };

const storeForm = (options) => ({
    action: store.url(options),
    method: 'post',
});
store.form = storeForm;

so <Form {...store.form()}> is <Form action="/login" method="post">
```

WAYFINDER

A build step reads Laravel's routes and writes one small function per route into resources/js/routes and resources/js/actions.

WHY BOTHER

Nobody types '/login' in a component. Rename the route in PHP and the build fails where the old name is used, before a user finds it.

THE @SEE LINE

It names the PHP method that answers: Fortify's login controller. That is where the kit's login() lives.

IN YOURS

The addresses are strings in one file, api.js. Same idea: one place.

auth.user: Laravel shares it with every page

LARAVEL app/Http/Middleware/HandleInertiaRequests.php

```php
public function share(Request $request): array
{
    return [
        ...parent::share($request),
        'name' => config('app.name'),
        'auth' => [
            'user' => $request->user(),
        ],
    ];
}
```

REACT components/nav-user.tsx · shortened

```tsx
export function NavUser() {
    const { auth } = usePage().props;

    if (!auth.user) {
        return null;
    }
    return <UserInfo user={auth.user} />;
}
```

SHARE()

Middleware that runs on every request. Whatever it returns is added to the props of every page. $request->user() is the same call as in your user() method.

USEPAGE()

A hook that returns the current page object. Any component can read the shared props from it. It plays the part of your useSession().

IN YOURS

session.jsx asks GET /api/user once, on load, and keeps the answer in state. One extra request; in return the API does not need to know about pages.

Log out: a link that sends a POST

STARTER KIT `resources/js/components/user-menu-content.tsx · shortened`

```
const handleLogout = () => {
    cleanup();
    router.flushAll();
};

<Link
    href={logout()}
    as="button"
    onClick={handleLogout}
>
    <LogOut className="mr-2" />
    Log out
</Link>
```

LOGOUT()

Another generated route function: { url: '/logout', method: 'post' }. Logging out changes something, so it is a POST, never a GET.

AS="BUTTON"

Inertia's <Link> draws a <button> and sends the POST itself, with the CSRF token.

ROUTER.FLUSHALL()

Throws away pages Inertia had fetched ahead and kept. The next person at this browser must not see them.

IN YOURS

await logout(); navigate('/'). setUser(null) does the forgetting.

SUMMARY

Part 4: every file you wrote has a starter-kit twin

THE JOB	STARTER KIT	YOURS
Start React	`resources/js/app.tsx`	`web/src/main.jsx`
Address to page	`routes/web.php, pages/ by name`	`web/src/App.jsx`
The frame	`resources/js/layouts/`	`web/src/components/Layout.jsx`
The login form	`pages/auth/login.tsx, <Form>`	`pages/AuthPages.jsx, useAuthForm`
Where to send it	`resources/js/routes/ (generated)`	`web/src/api.js`
Who is logged in	`HandleInertiaRequests, usePage()`	`GET /api/user, session.jsx`
Check the password	`Fortify (a package)`	`AuthController::login()`

Same jobs, same rules, same session cookie. Read the starter kit to see how the Laravel team writes each piece; build yours from the pantry.

Commit today. Homework 9 finishes it by Fri 11/20.

1	**The login, on the API**	Files 1 to 5. check-login.sh passes 9 of 9.	**Today · Part 2**
2	**The login, on the pages**	Files 6 to 9. Register, log in, log out; messages next to the fields.	**Today · Part 3**
3	**The write endpoints**	Your three writes stop answering 501: 401, 422, then 201, 200, 204.	**Homework 9**
4	**One policy**	Who may edit and who may delete. The wrong user gets 403.	**Homework 9**
5	**PROCESS.md**	Who did which step, what broke, what is blocked.	**Homework 9**

There is no autograder for Homework 9. Section 4 of the homework page lists what your app should do, in the order the grader tries it. Do the list yourselves on a fresh clone before you submit.

ONE LOGIN, TWO SHAPES

Read one that works. Then build yours.

The starter kit shows how the Laravel team writes a login.

The pantry shows it in your project's shape. The rules are the same in both.